LA_2
PROJECT-BASED LEARNING (PBL) REPORT

Aim of experiment: To select two modules of the AI-Powered Civic Complaint Management System, implement and test them, and manage the project using JIRA with defect tracking.

Author: Meenakshi Motwani

Section / Roll No: A5_B2 / 19

Date of experiment: 30/10/2026

Case study: AI-POWERED CIVIC COMPLAINT MANAGEMENT SYSTEM FOR MUNICIPAL CORPORATIONS.

Problem statement: To develop a centralized system for citizen civic grievance redressal, automating complaint intake, municipal officer assignment, real-time lifecycle tracking, and transparent chronological audit logging.

1. SELECTED MODULES

Two modules were selected for implementation. Module 1 is Complaint Submission (SRS section 4.1, Feature 2). Module 2 is Complaint Tracking (SRS section 4.2, Feature 4). Module 2 is the core function of the system, and it depends on Module 1 because tracking needs a registered complaint with an assigned officer. This dependency is how the two modules integrate. Officer and administrator data needed for the demonstration is provided as sample data.

2. MODULE 1: COMPLAINT SUBMISSION

This module lets registered citizens log in and submit a civic complaint with a title, description, category, priority, and location (between 3 and 200 characters), and attach an optional supporting image (PNG, JPG, JPEG, GIF, WEBP, up to 5MB). The system validates the input, auto-generates a unique Complaint ID, creates the initial status history audit record, and automatically assigns the complaint to the municipal officer. Once submitted, the complaint becomes eligible for departmental action and tracking.

Table 1: Requirements implemented in Module 1

Requirement	Description	Implemented by
REQ-CS-01	Authorized citizens can register and submit complaints	Registration and login with session auth
REQ-CS-02	Store complaint details, location, category, and priority	complaints table and Complaint ORM model
REQ-CS-03	Attach supporting image (PNG/JPG/JPEG/GIF/WEBP, max 5MB)	File upload handling with allowed_file()

REQ-CS-04	Validate mandatory fields and location length (3–200 chars)	Server-side input validation rules
REQ-CS-05	Auto-generate Complaint ID and initial status history log	Auto-increment ID and StatusHistory table

3. MODULE 2: COMPLAINT TRACKING

This module allows citizens to track the progress of their submitted complaints. Citizens can search by numeric Complaint ID, search by title keyword, and filter complaints by status. Opening a complaint displays the full details, supporting image, a 6-stage lifecycle progress stepper, and a chronological status history timeline. Ownership isolation ensures citizens only view their own complaints. Municipal officers can review assigned complaints and update their status to In Progress or Complaint Resolved with remarks, which immediately updates the citizen's tracking page.

Table 2: Requirements implemented in Module 2

Requirement	Meaning	Implemented by
REQ-CT-01	Citizens view only their own complaints (ownership isolation)	Session user_id query filter
REQ-CT-02	Search by Complaint ID and title keyword	Search query parsing in routes/citizen.py
REQ-CT-03	Filter complaints by status	Status dropdown filter query
REQ-CT-04	Display 6-stage lifecycle progress stepper and details	complaint_detail.html progress stepper
REQ-CT-05	Show chronological status history timeline with remarks	status_history table ordered by timestamp
REQ-CT-06	Officer reviews and updates status with remarks	Officer dashboard and update-status route

Design limitations: one municipal officer is auto-assigned per complaint in this prototype, categorization uses rule-based keyword matching, and citizen satisfaction rating is kept for future sprints.

4. TECHNOLOGY STACK

The system is a web application built with Python and Flask for the backend, SQLite as the relational database, and Jinja2 with Bootstrap 5 for the user interface. Passwords are hashed with Werkzeug security, login sessions use Flask session cookies, and unit tests use Python's built-in unittest framework. This stack meets the SRS needs for a web application, a relational database, role-based access control and input validation.

5. JIRA PROJECT MANAGEMENT

The project was planned and tracked in a JIRA project named Civic Complaint System (key CIVIC). Two epics were created from the SRS feature groups. Modules Complaint Submission and Complaint Tracking were developed. User stories were written in the 'As a ... I want ... so that ...' form with Fibonacci story points (1, 2, 3, 5, 8), assigned to team members, and tracked across sprints.

5.1 Epics, Backlog and User Stories

All issues were created with detailed descriptions, parent epics, story points, and broken into sub-tasks. Status was transitioned from To Do to In Progress to Done as development was carried out.

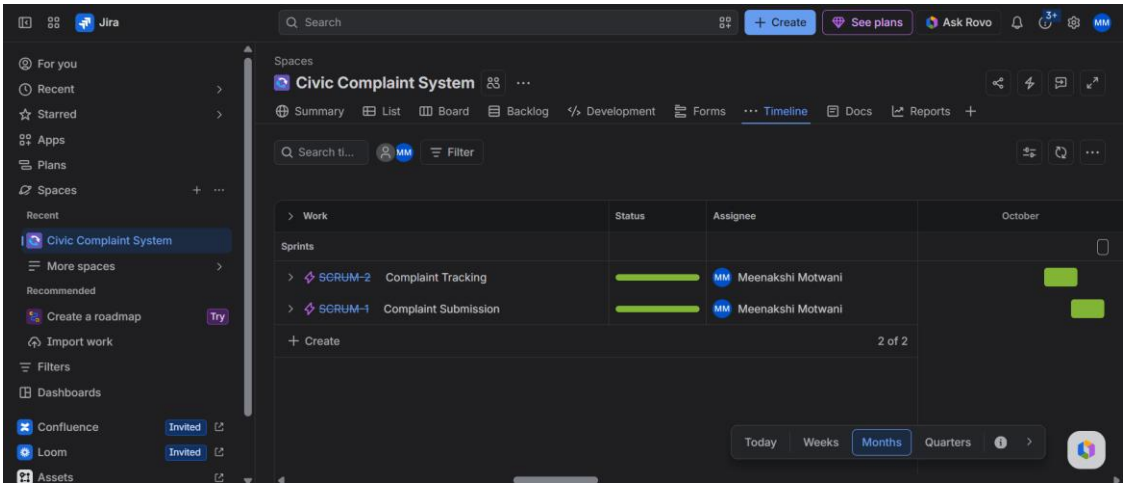
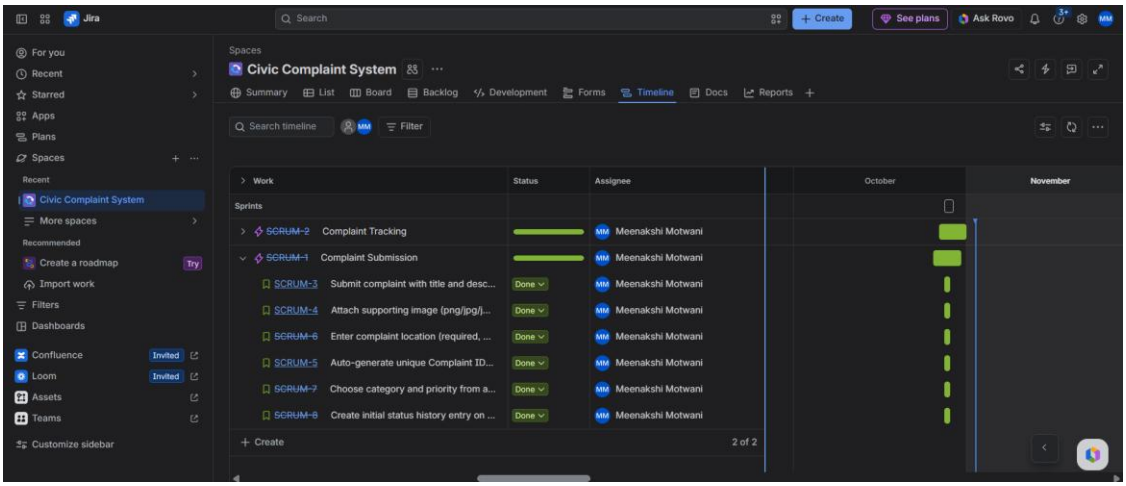


Figure 1: Epics created in the JIRA project (CIVIC-1 to CIVIC-6)



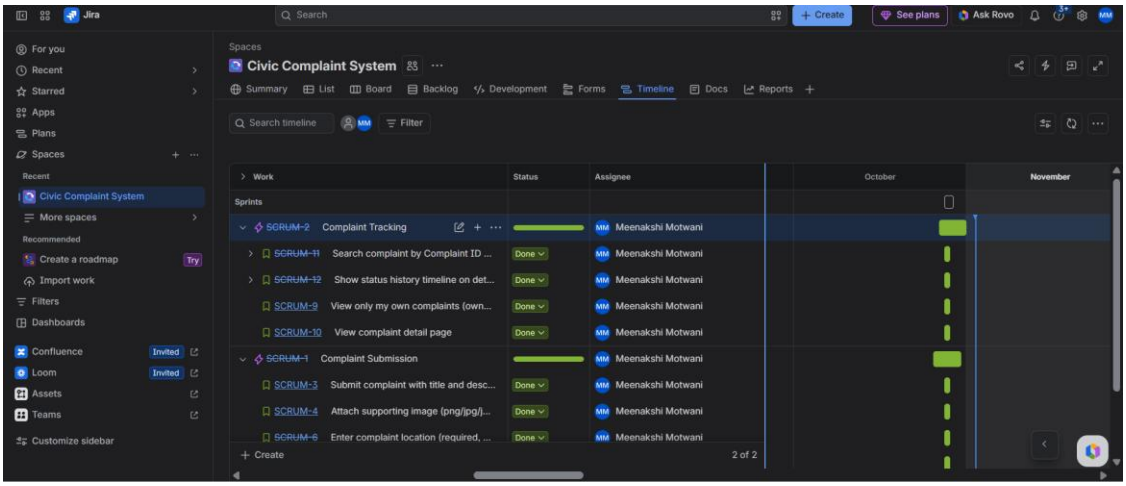


Figure 2: Product backlog with the stories planned for Sprint 1 and Sprint 2

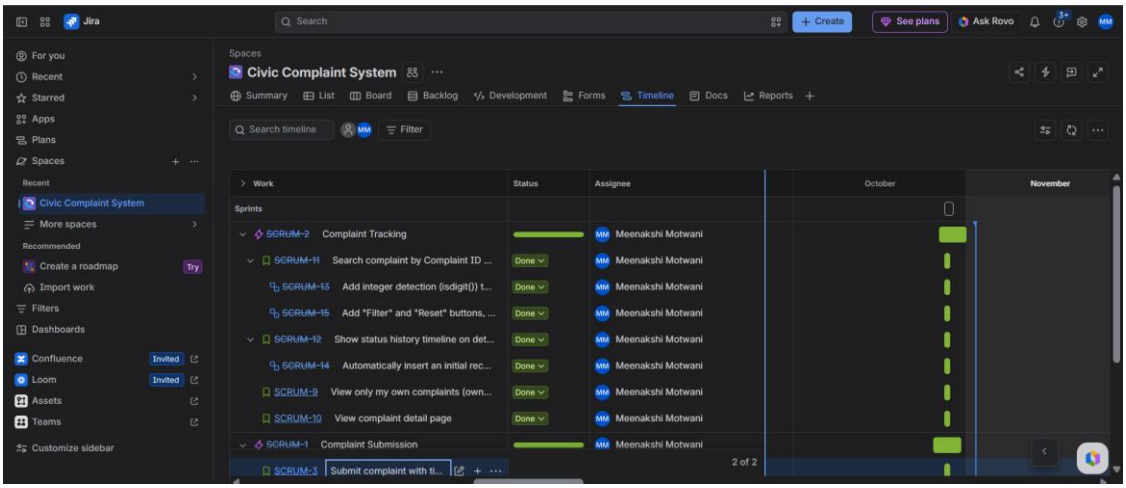


Figure 3: User story 13, 14 and 15 with its parent epic

5.2 Sprint 1

Sprint 1 covered Complaint Submission (registering complaints, location validation, image upload, and officer auto-assignment).

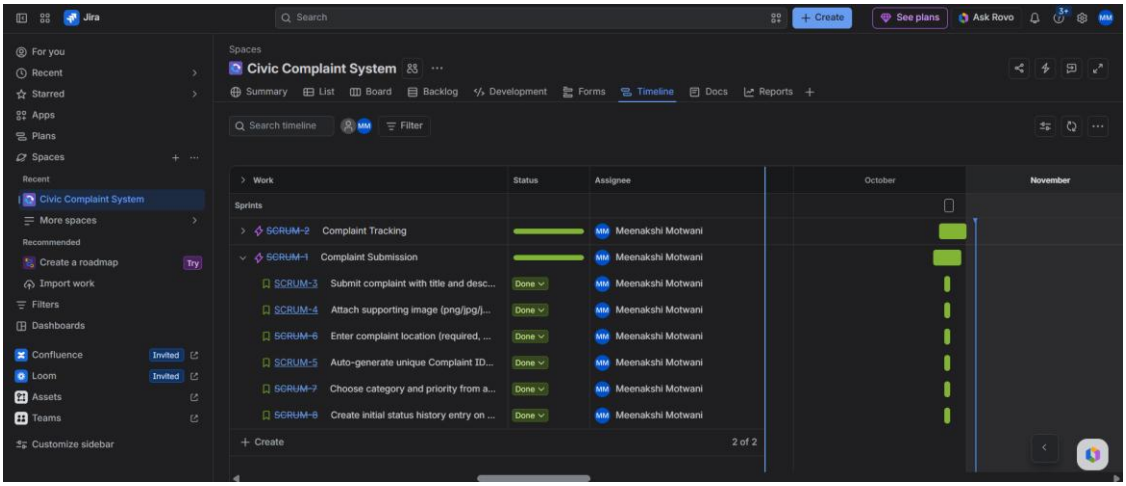


Figure 4: Project summary with the status overview after Sprint 1

5.3 Sprint 2

Sprint 2 covered Complaint Tracking: viewing complaints, search by numeric ID, status filtering, the 6-stage lifecycle progress stepper, chronological audit history timeline, and municipal officer status updates. Testing of Module 2 was carried out in this sprint. Two bugs [BUG-01 and BUG-02] were identified, solved, and documented under the sprint.

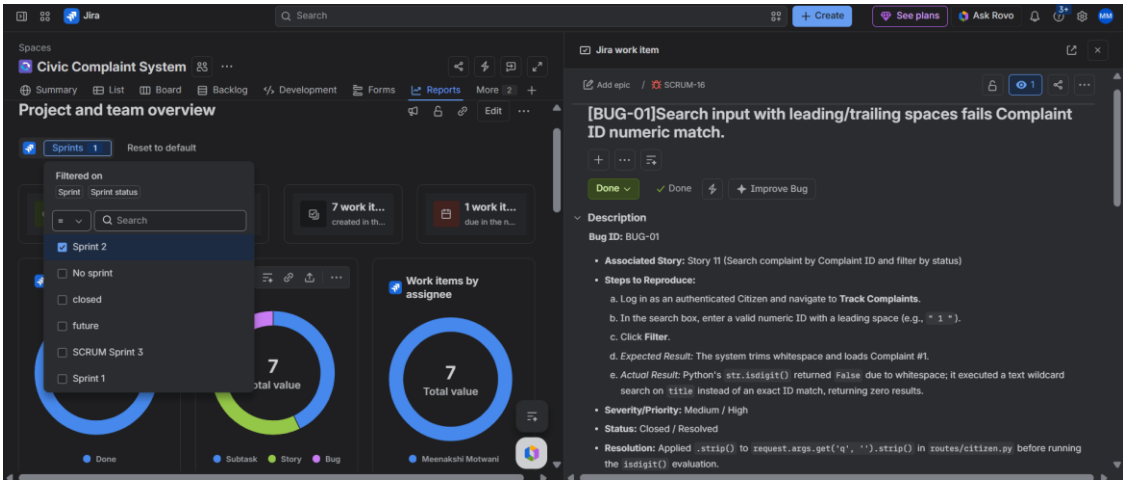


Figure 5: Bug 1

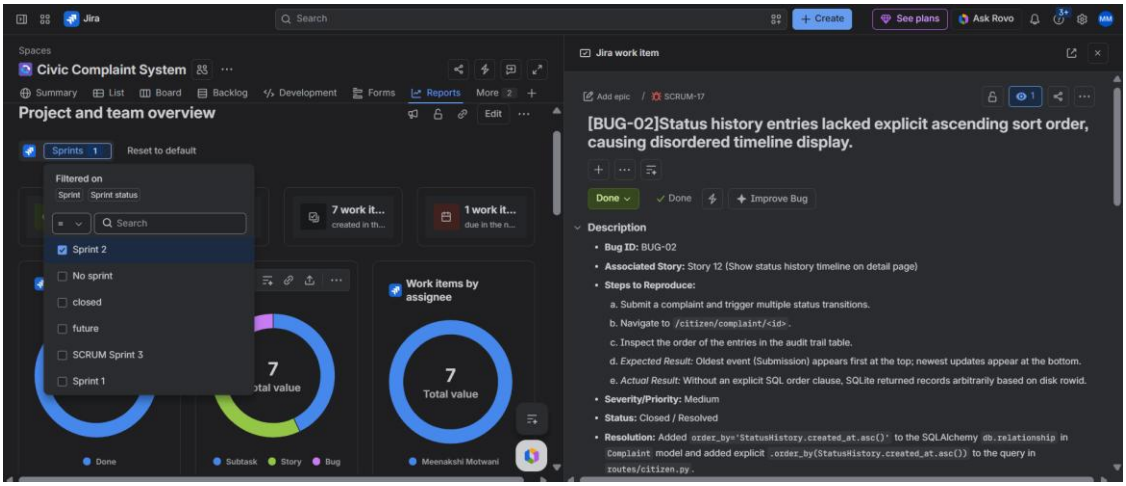


Figure 6: Bug 2

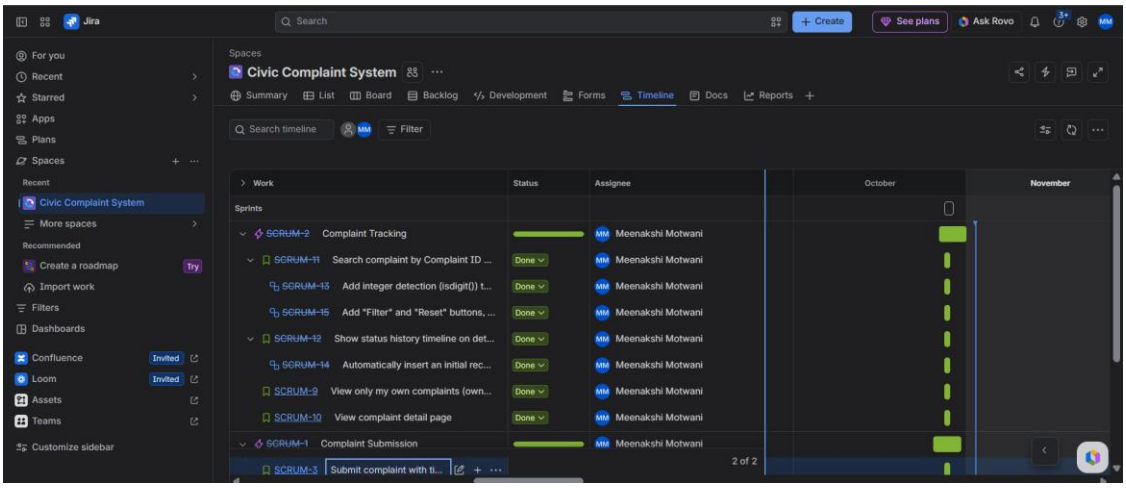


Figure 7: Completing Sprint 2 (all work items completed)

6. DATABASE DESIGN

The data is stored in a SQLite database named civic_complaints.db with three tables. The users table holds login accounts with hashed passwords and roles (Citizen, Municipal Officer, Administrator). The complaints table holds complaint details, location, category, priority, status, and foreign keys for the citizen author and assigned officer. The status_history table logs each status transition chronologically with timestamps and remarks. Foreign keys maintain relational integrity, and email addresses are unique.

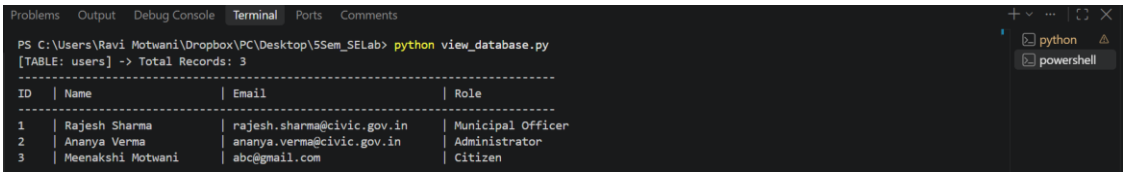


Figure 8: Structure of the users table

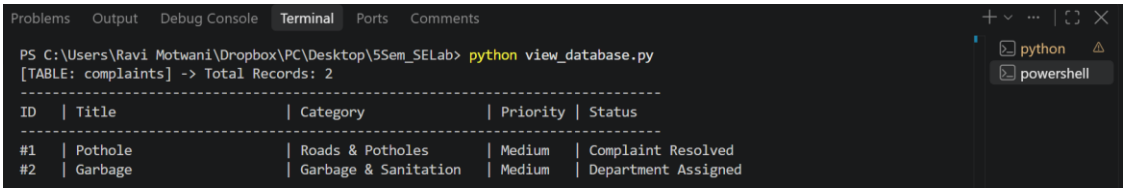


Figure 9: Structure of the complaints table

```
[TABLE: status_history] -> Total Records: 7
--------------------------------------------------------------------------------
CID  | Status                | Updated By            | Remarks
--------------------------------------------------------------------------------
#1   | Complaint Submitted   | Meenakshi Motwani     | Complaint registered with s
#1   | Department Assigned   | AI Assignment Service | Automatically assigned to M
#1   | In Progress           | Rajesh Sharma         | Crew DIspatched
#1   | Needs More Info       | Rajesh Sharma         | Location insufficient
#1   | Complaint Resolved    | Rajesh Sharma         | Status updated to Complaint
#2   | Complaint Submitted   | Meenakshi Motwani     | Complaint registered with s
#2   | Department Assigned   | AI Assignment Service | Automatically assigned to M
================================================================================
PS C:\Users\Ravi Motwani\Dropbox\PC\Desktop\5Sem_SELab>
```

Figure 10: Structure of the status_history table

7. MODULE 1 IMPLEMENTATION

Citizens log in, open Submit complaint, enter the complaint details, location and priority, optionally attach an image, and submit the form. The system validates the input, auto-generates a unique Complaint ID, stores the complaint, and displays a success message. The validation rules are as follows:

• Title and description: required, non-empty text.

• Location: required, between 3 and 200 characters.

• Category and priority: must be one of the allowed dropdown values.

• Image upload: optional, allowed formats are PNG, JPG, JPEG, GIF, WEBP, maximum 5MB.

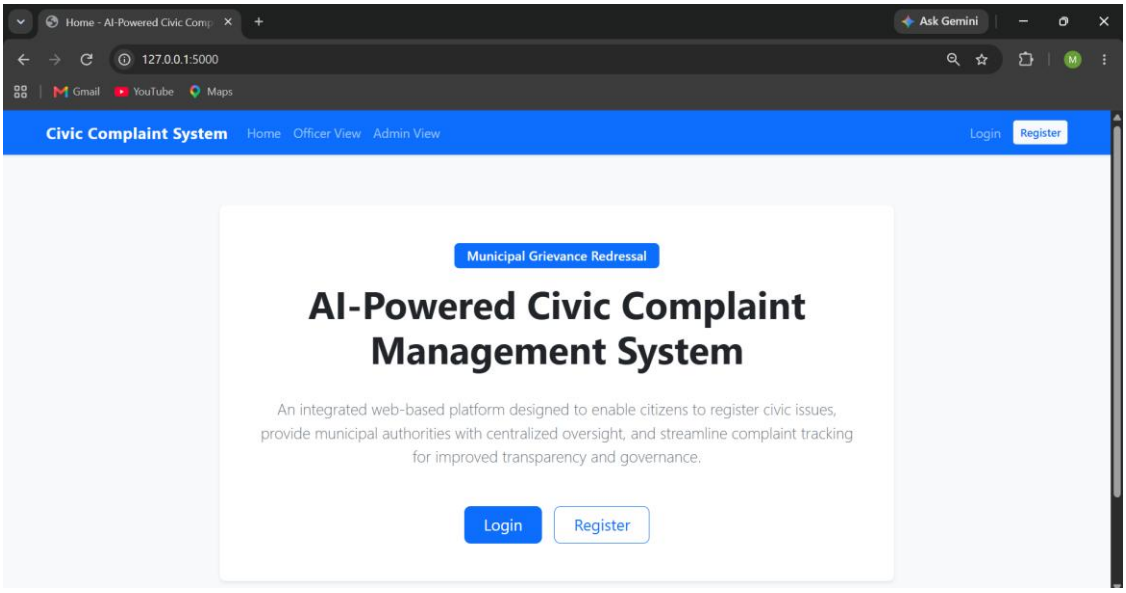


Figure 11: Login page

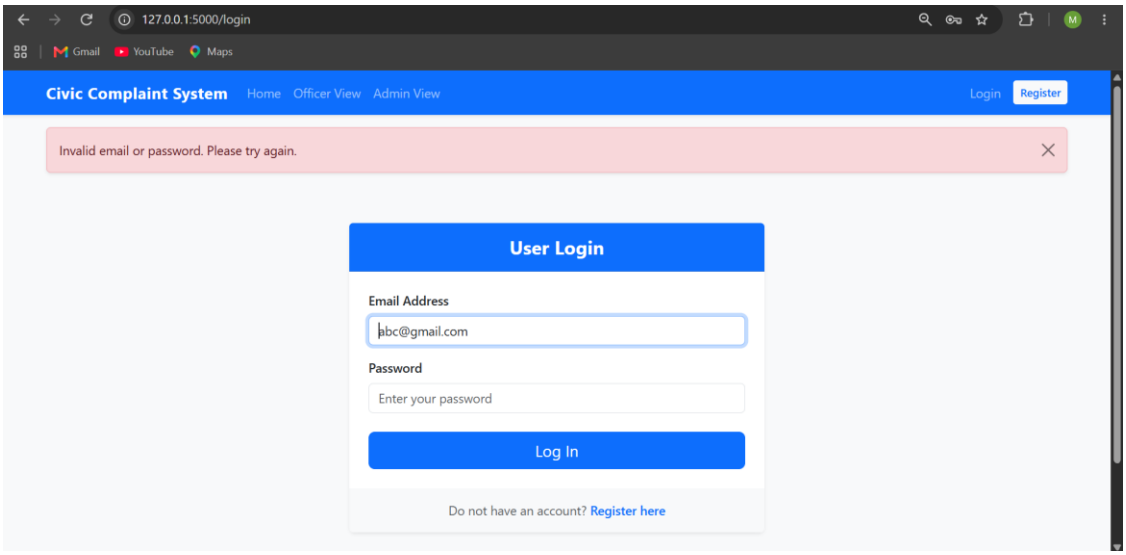


Figure 12: Error message for an incorrect username or password

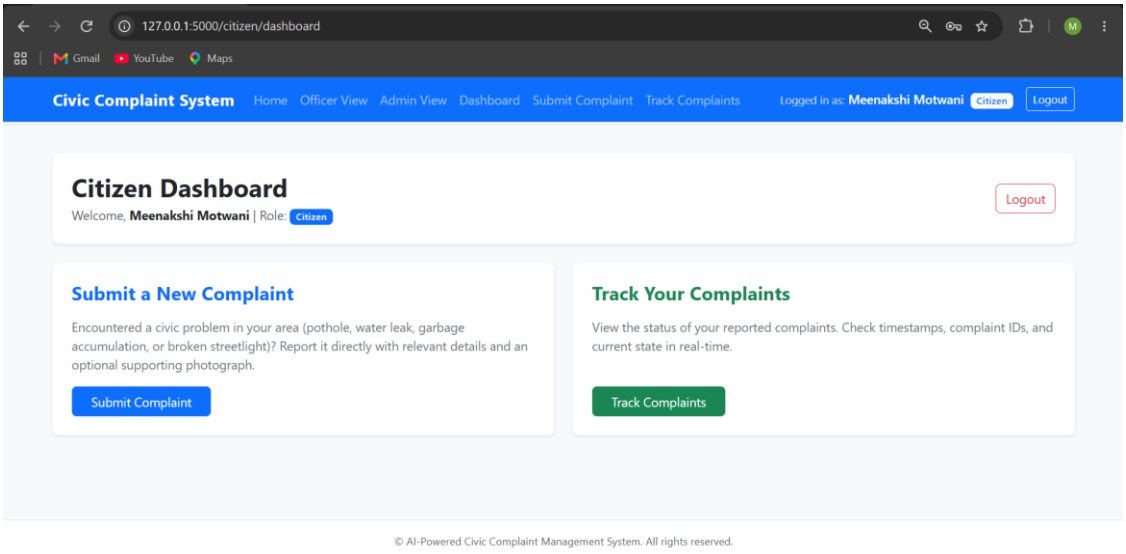


Figure 13: Home Page for citizen

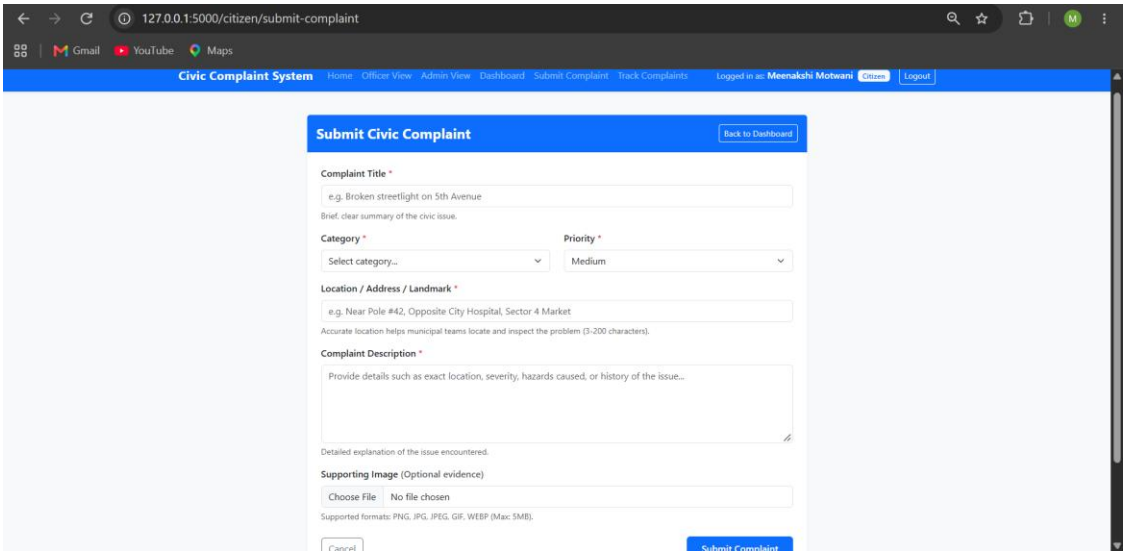


Figure 14: Empty complaint registration form

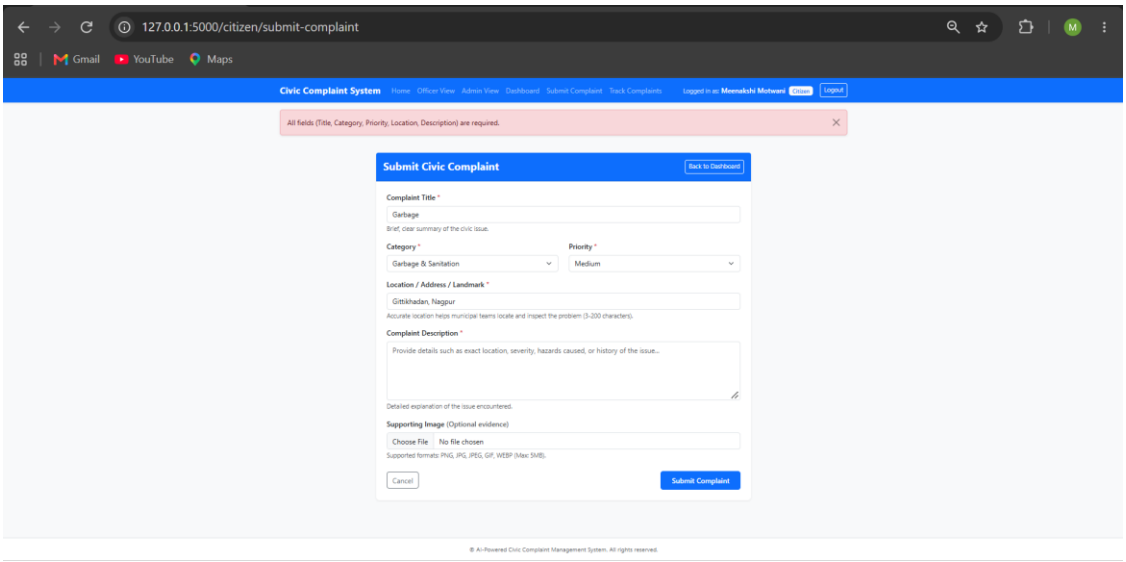


Figure 15: Validation errors shown for missing fields

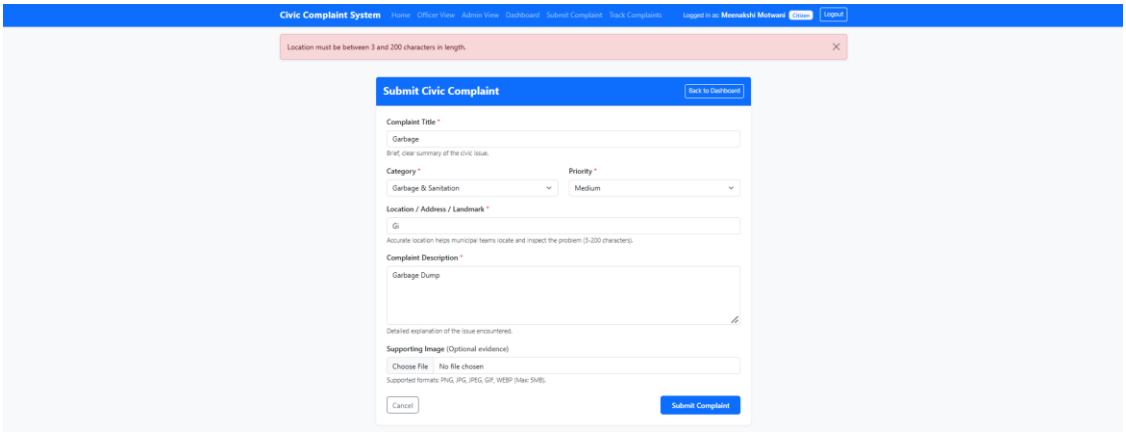


Figure 16: Validation error shown for location shorter than 3 characters

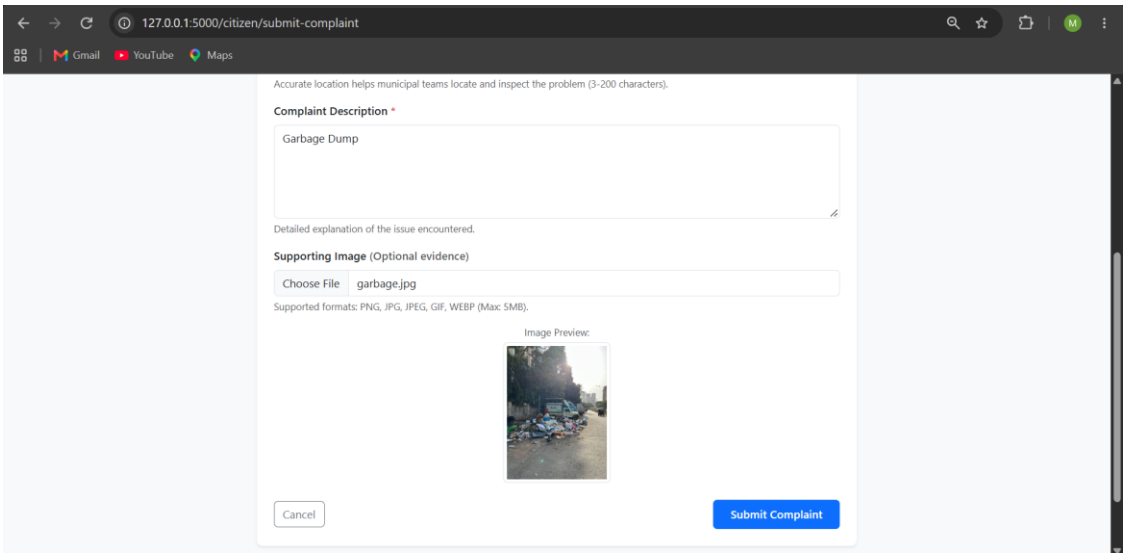


Figure 17: Live image preview before submission

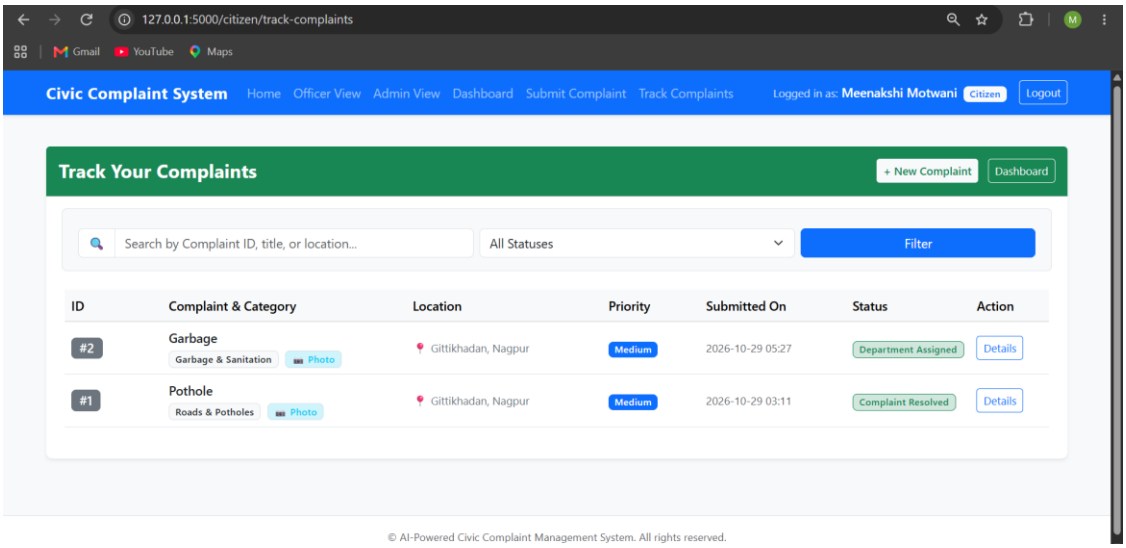


Figure 18: Registered complaint stored in the complaints table

8. MODULE 2 IMPLEMENTATION

Citizens open the Track complaints page to view their submitted grievances with color-coded status badges. They can search by Complaint ID or keyword, and filter by status. Clicking View Details opens the complaint detail page, showing full metadata, uploaded evidence, a 6-stage progress stepper, and the status history audit trail. Accessing another citizen's complaint returns HTTP 403 Forbidden. Municipal officers open the Officer dashboard, view assigned complaints, and update the status (In Progress, Complaint Resolved) with remarks, which immediately updates the citizen's tracker.

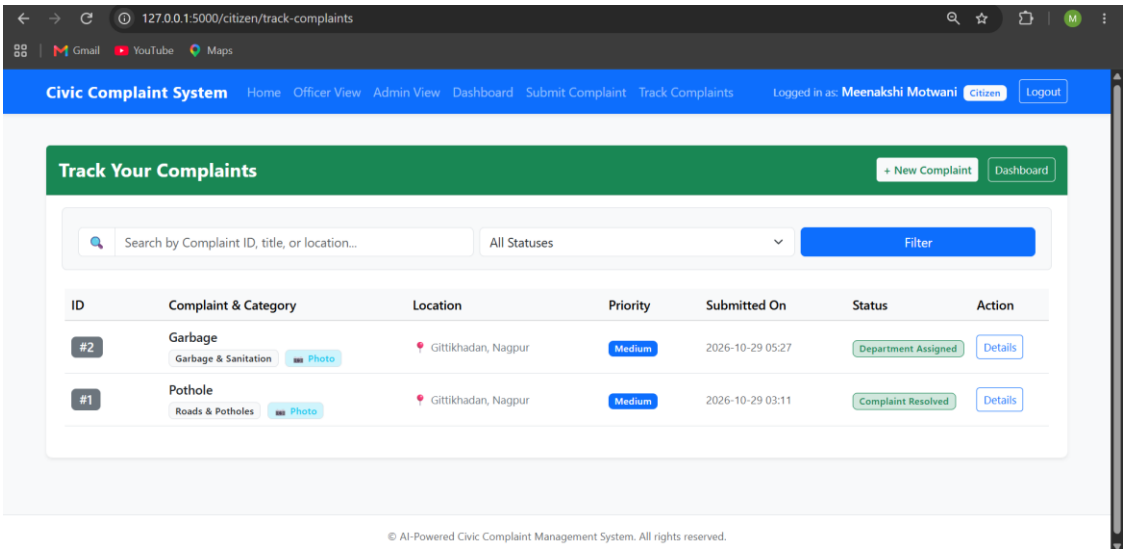


Figure 19: Citizen tracking dashboard listing complaints with status badges

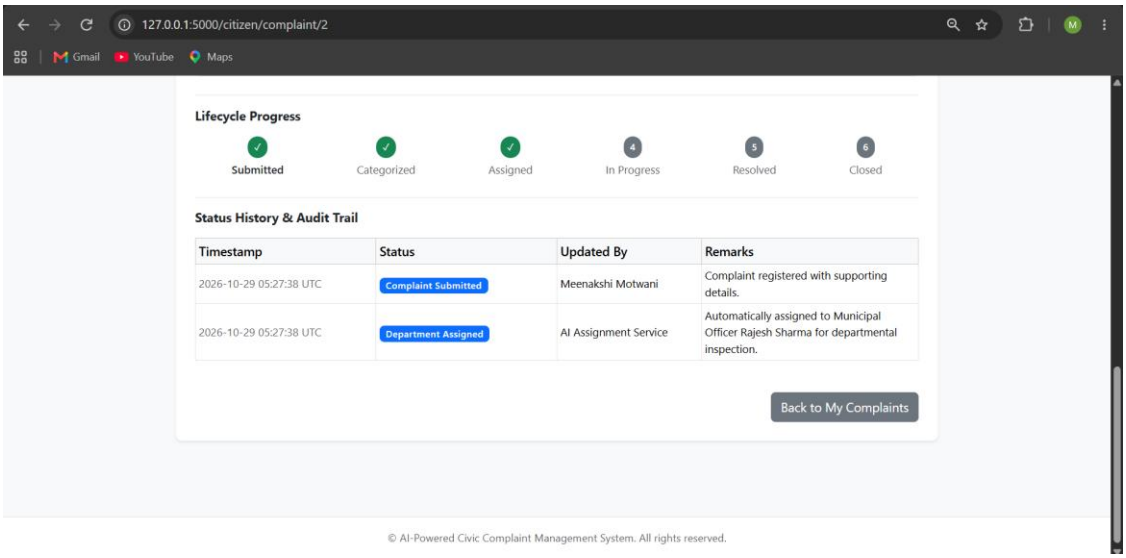


Figure 20: Complaint detail view showing metadata and 6-stage progress stepper

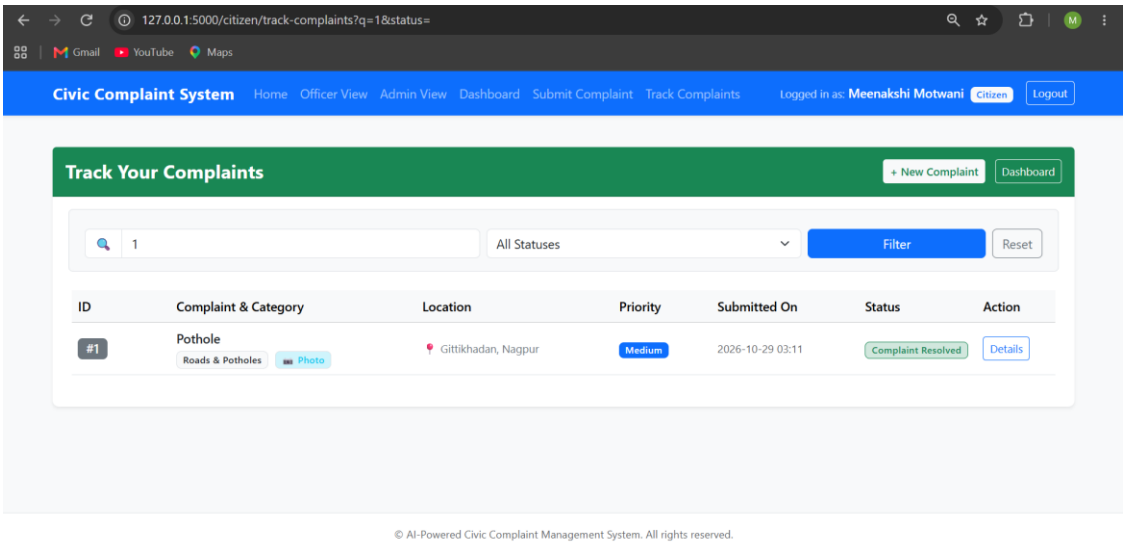


Figure 21: Filtered search by numeric Complaint ID

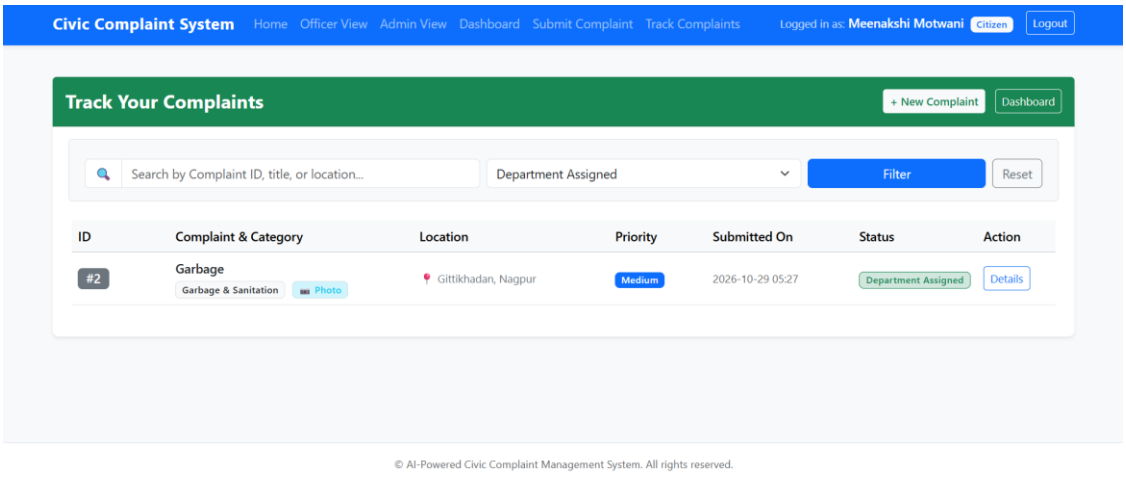


Figure 22: Filtered search by status dropdown ('Department Assigned')

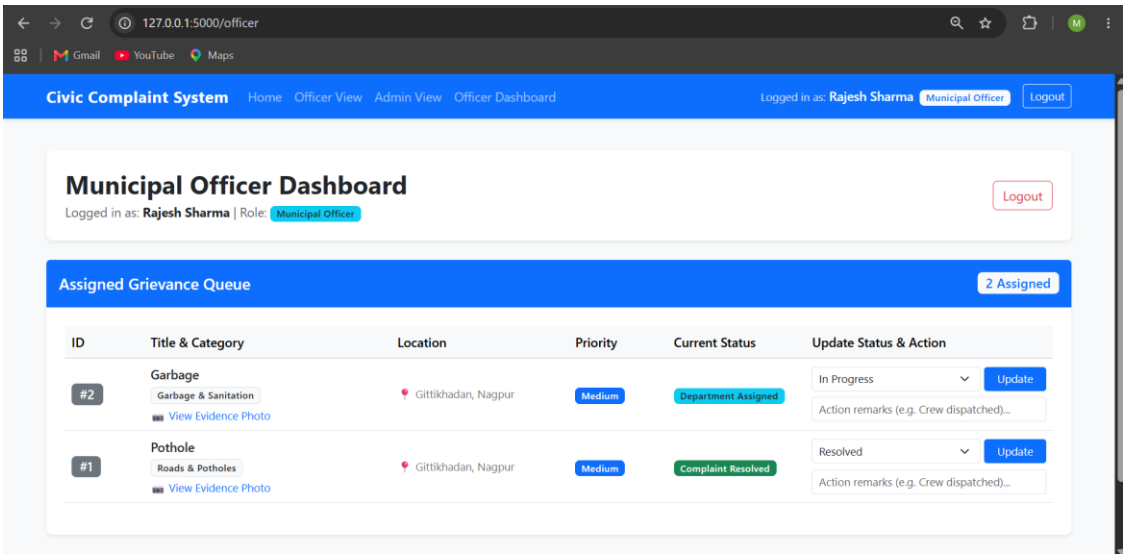


Figure 23: Municipal Officer dashboard showing assigned complaints queue

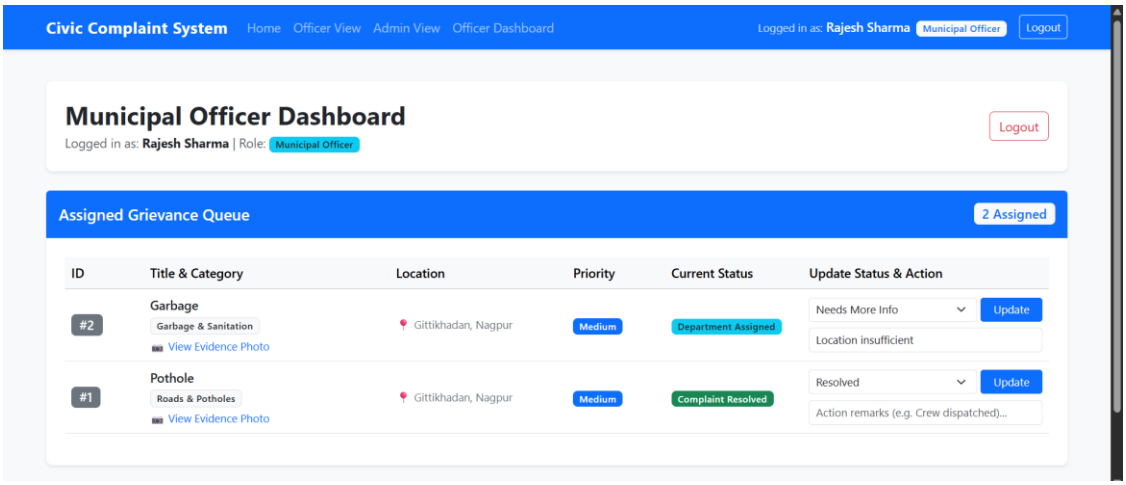


Figure 24: Municipal Officer updating status to Needs More Info with remarks

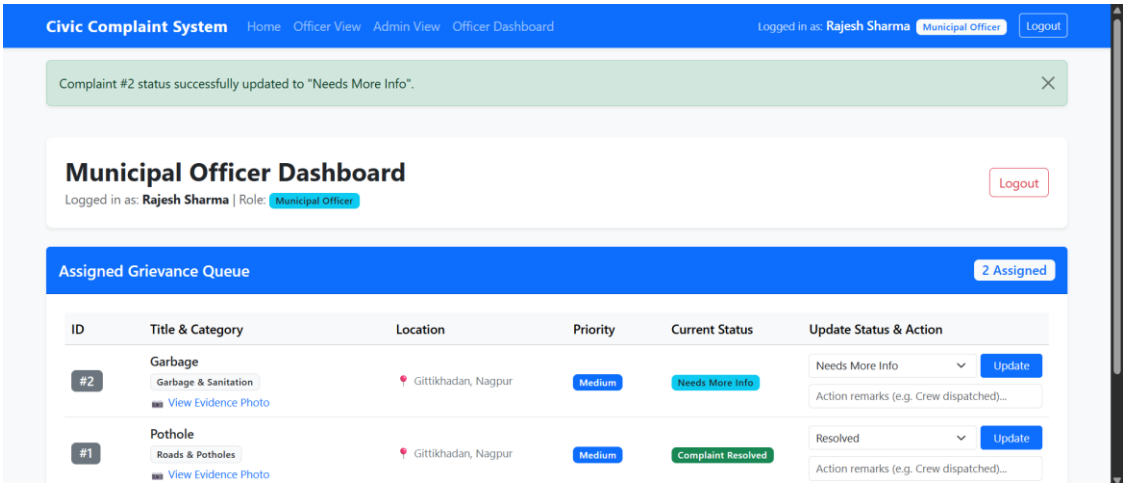


Figure 25: Updated progress stepper reflecting officer action

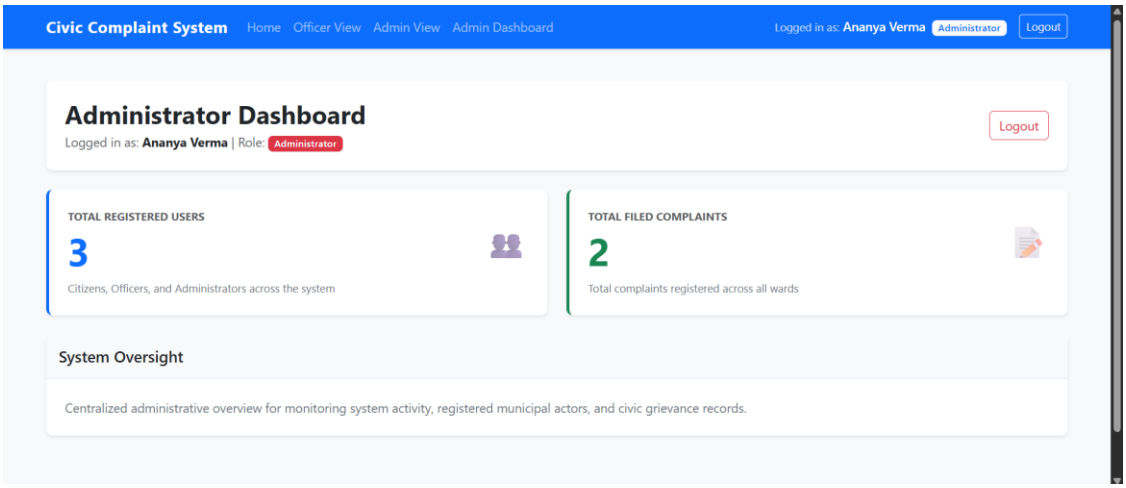


Figure 26: Administrator dashboard displaying user counts and complaint volume

9. SOURCE CODE

The project follows a layered MVC structure: routes handle web requests and role-based access control, models define database schemas and password hashing, and Jinja2 templates render the views. The source code is kept in a Git repository.

Table 3: Main files of the project

File	Purpose
app.py	Starts the server, registers Blueprints, error handlers, and creates tables
config.py	Configuration settings: database URI, 5MB upload limit, allowed formats
models/user.py	User model, password hashing with Werkzeug, and role definitions
models/complaint.py	Complaint and StatusHistory ORM models with foreign keys
routes/auth.py	Registration, login verification, session management, and logout
routes/citizen.py	Complaint submission, validation, auto-assignment, tracking and search
routes/officer.py	Municipal officer dashboard and status transition route with remarks
routes/admin.py	Administrator dashboard displaying platform metrics
templates/	Jinja2 templates for base layout, submission, tracking, and officer view
run_comprehensive_tests.py	Automated test suite executing 18 tests and generating Excel report

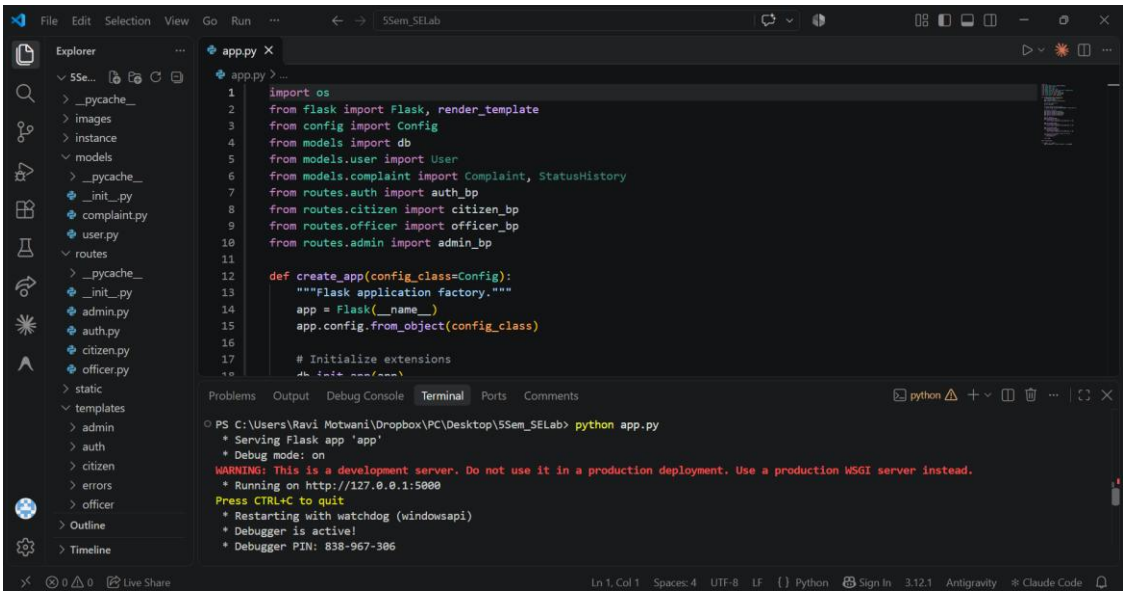


Figure 27: Project structure in VS Code, with the server started from the terminal

10. TESTING

Both modules were tested using functional testing, validation testing, boundary value analysis, equivalence partitioning, unit testing and integration testing. The test cases follow the format Test Case ID, Module, Test Scenario, Test Data, Expected Result, Actual Result and Status.

10.1 Module 1 Test Cases

Table 4 lists the 10 test cases for Module 1. Validation tests checked empty inputs, boundary lengths, and file extensions. All test cases passed.

Table 4: Test cases for Module 1

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC-01	Auth	Valid citizen login	ravi@civic.gov.in / pass123	Dashboard opens	Dashboard opened	**Pass**
TC-02	Auth	Wrong password	ravi@civic.gov.in / wrong	Error: 'Invalid email or password'	Error message displayed	**Pass**
TC-03	Submission	Valid complaint submission	Title, Cat, Prio, Location	Complaint saved; ID #1 generated	Saved; ID #1 generated	**Pass**
TC-04	Submission	Empty mandatory fields	All fields blank	Error shown on fields	Validation returned error	**Pass**
TC-05	Submission	Location 2 chars (boundary)	Location='AB'	Error: minimum 3 chars	Error: minimum 3 chars	**Pass**
TC-06	Submission	Location 3	Location='ABC'	Accepted and	Accepted and	**Pass**

		chars (boundary)		saved	saved	
TC-07	Submission	Location 200 chars (boundary)	200 characters	Accepted and saved	Accepted and saved	**Pass**
TC-08	Submission	Disallowed file (.exe)	file='script.exe'	Error: 'Invalid file format'	Error displayed; rejected	**Pass**
TC-09	Submission	Valid image (.png)	file='photo.png'	Saved in static/uploads/	Saved in static/uploads/	**Pass**
TC-10	Unit	allowed_file helper	test.jpg, test.exe	True for jpg, False for exe	True for jpg, False for exe	**Pass**

10.2 Module 2 Test Cases

Table 5 lists the 8 test cases for Module 2. Tests evaluated complaint tracking, numeric ID search, status dropdown filtering, timeline display, and cross-citizen access prevention (IDOR). All test cases passed.

Table 5: Test cases for Module 2

Test Case ID	Module	Test Scenario	Test Data	Expected Result	Actual Result	Status
TC-11	Tracking	View complaint detail page	Complaint ID #1	Metadata and stepper rendered	Metadata and stepper rendered	**Pass**
TC-12	Tracking	Search by exact Complaint ID	query q='1'	Only Complaint #1 returned	Complaint #1 isolated	**Pass**
TC-13	Tracking	Search with whitespace	query q=' 1 '	Whitespace stripped; #1 found	Complaint #1 found	**Pass**
TC-14	Tracking	Filter by status	status='In Progress'	Only matching complaints shown	Filtered table displayed	**Pass**
TC-15	Tracking	Cross-citizen isolation (IDOR)	Citizen A visits /complaint/2	HTTP 403 Forbidden abort	HTTP 403 returned; blocked	**Pass**
TC-16	Tracking	Chronological audit timeline	Inspect status_history rows	Ordered by timestamp asc	Ordered chronologically	**Pass**
TC-17	Officer	Officer updates status	new_status='In Progress'	Status updated; remark saved	Updated; audit row saved	**Pass**
TC-18	Integration	Submit then track flow	Citizen submits, opens track	New complaint in tracking table	New complaint in table	**Pass**

10.3 Testing Techniques Used

• Functional testing: login, complaint submission, tracking, and officer status update flows (TC-01, TC-03, TC-11, TC-17, TC-18).

• Validation testing: blank inputs and invalid credentials (TC-02, TC-04).

• Boundary value analysis: location length of 2, 3, and 200 characters (TC-05, TC-06, TC-07).

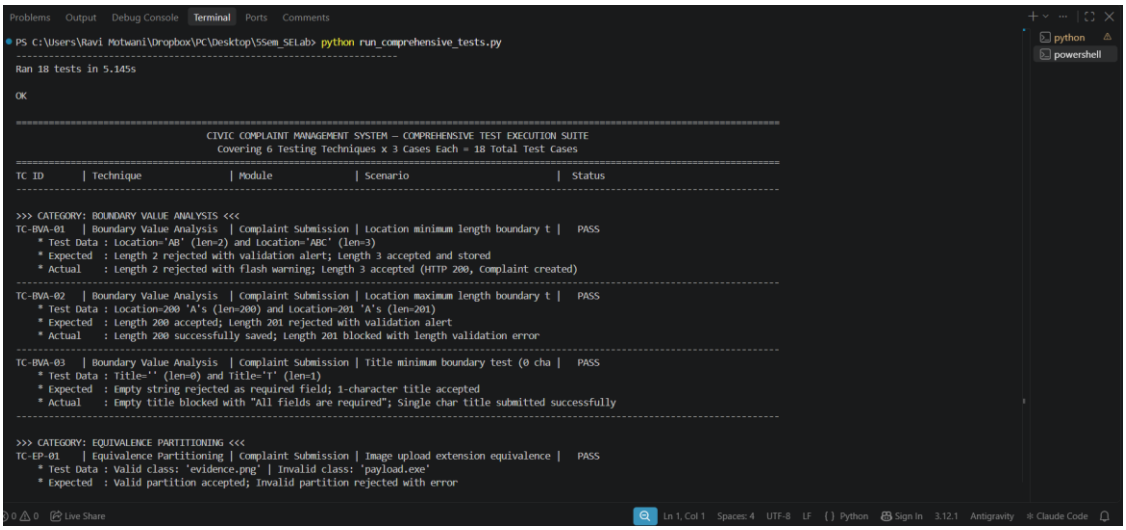
• Equivalence partitioning: valid PNG images versus disallowed EXE binaries (TC-08, TC-09).

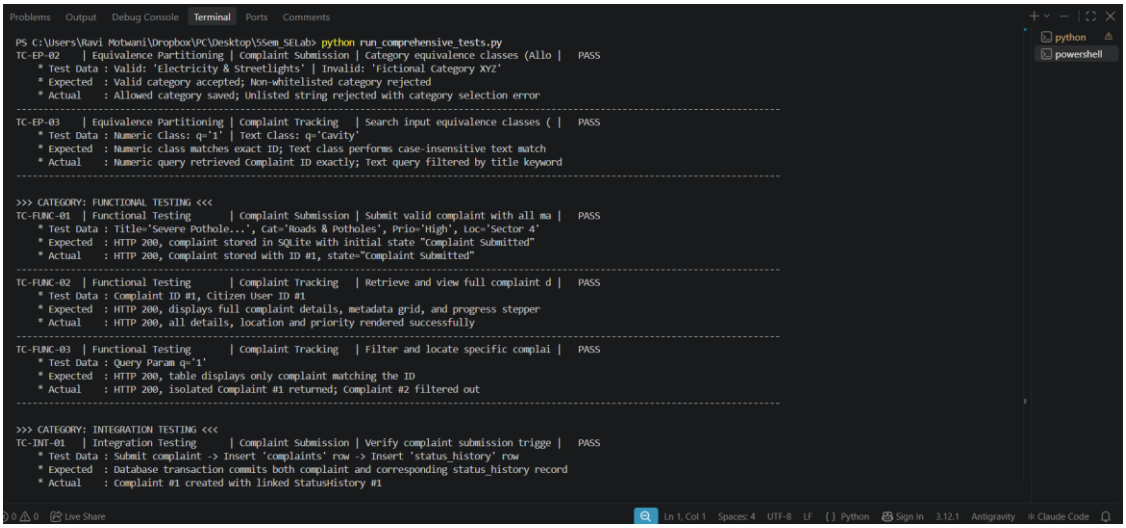
• Unit testing: file extension helper function and ORM default values (TC-10).

• Integration testing: submitting a complaint, auto-assigning an officer, and tracking its lifecycle (TC-15, TC-16, TC-18).

10.4 Unit Test Results

All 18 automated test cases passed using Python's unittest test runner in run_comprehensive_tests.py.





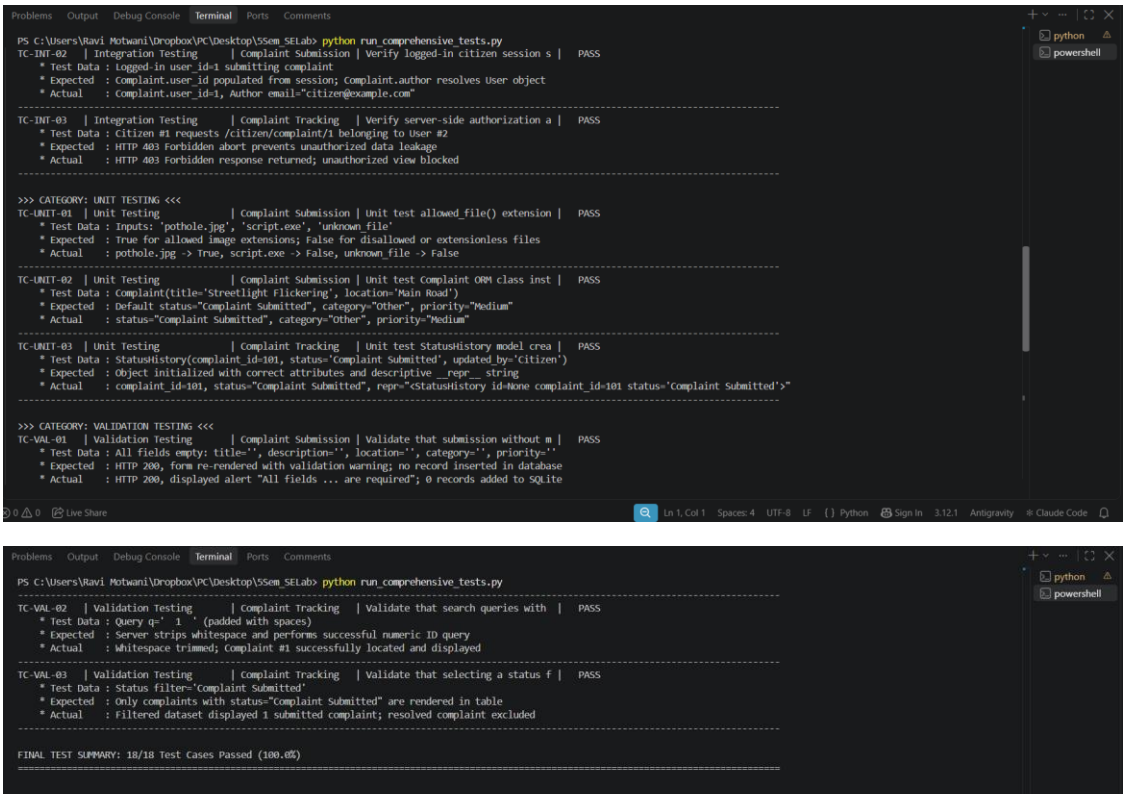


Figure 28: Output of run_comprehensive_tests.py showing 18 passing tests

11. DEFECTS FOUND AND FIXED

Defects were identified during testing, logged in JIRA, fixed, retested and closed. A test case originally revealed a defect where search inputs containing leading or trailing whitespace failed to match numeric Complaint IDs. The defect was logged as BUG-01, resolved by adding string whitespace stripping, and retested successfully.

Table 6: Details of defect SCRUM-16 (Search Whitespace)

Field	Details
Bug ID	SCRUM-16 (BUG-01)
Summary	Search input with leading/trailing spaces fails Complaint ID numeric match
Description	When a user enters a search query with accidental spaces (e.g. ' 1 '), isdigit() returns False because spaces are non-digits, falling back to title search and missing the complaint.
Steps to reproduce	1. Open http://127.0.0.1:5000/citizen/track-complaints. 2. In search box, enter ' 1 '. 3. Click Search.
Expected result	Whitespace is trimmed; Complaint #1 is retrieved.
Actual result	Search failed to match numeric ID; showed 'No complaints found'.
Severity / Priority	Medium

Status	Done (closed)
Resolution	Fixed: added q = request.args.get('q', '').strip() in routes/citizen.py prior to checking isdigit().

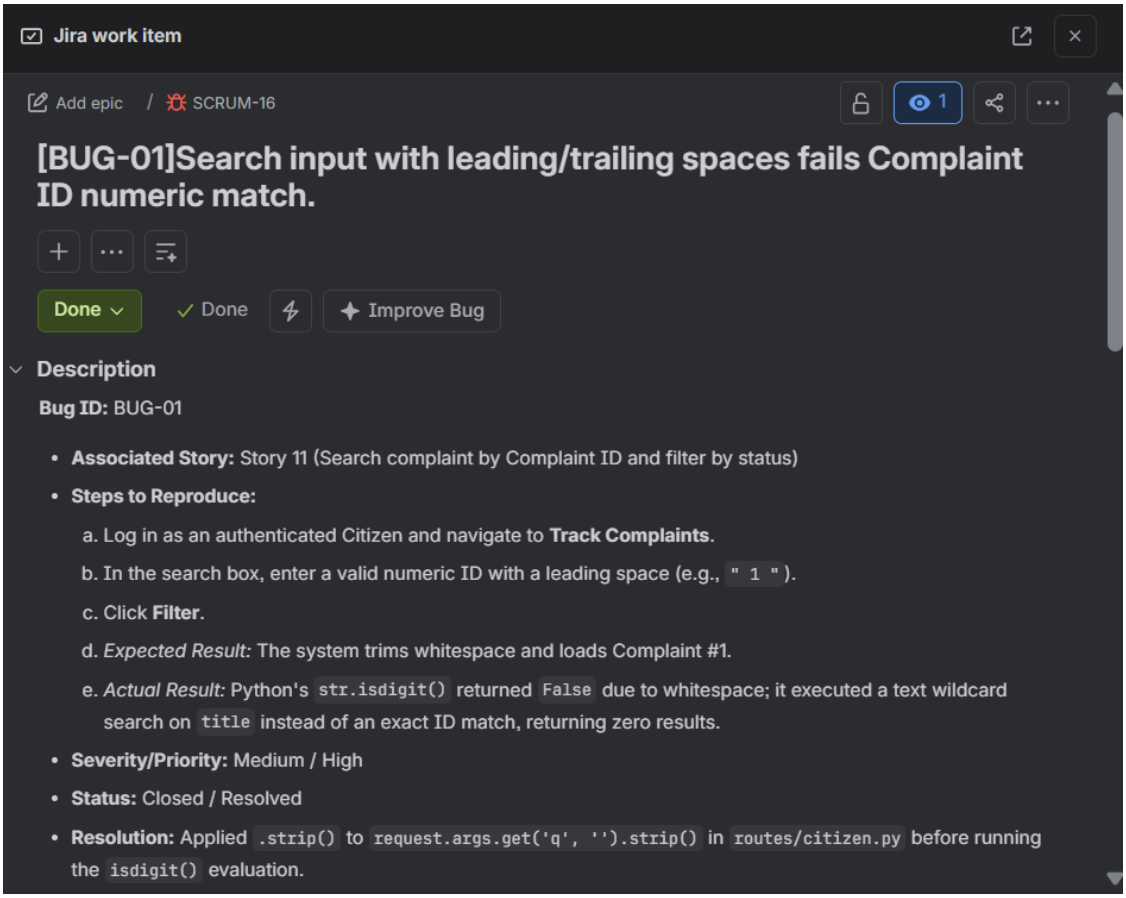


Figure 29: Bug 1

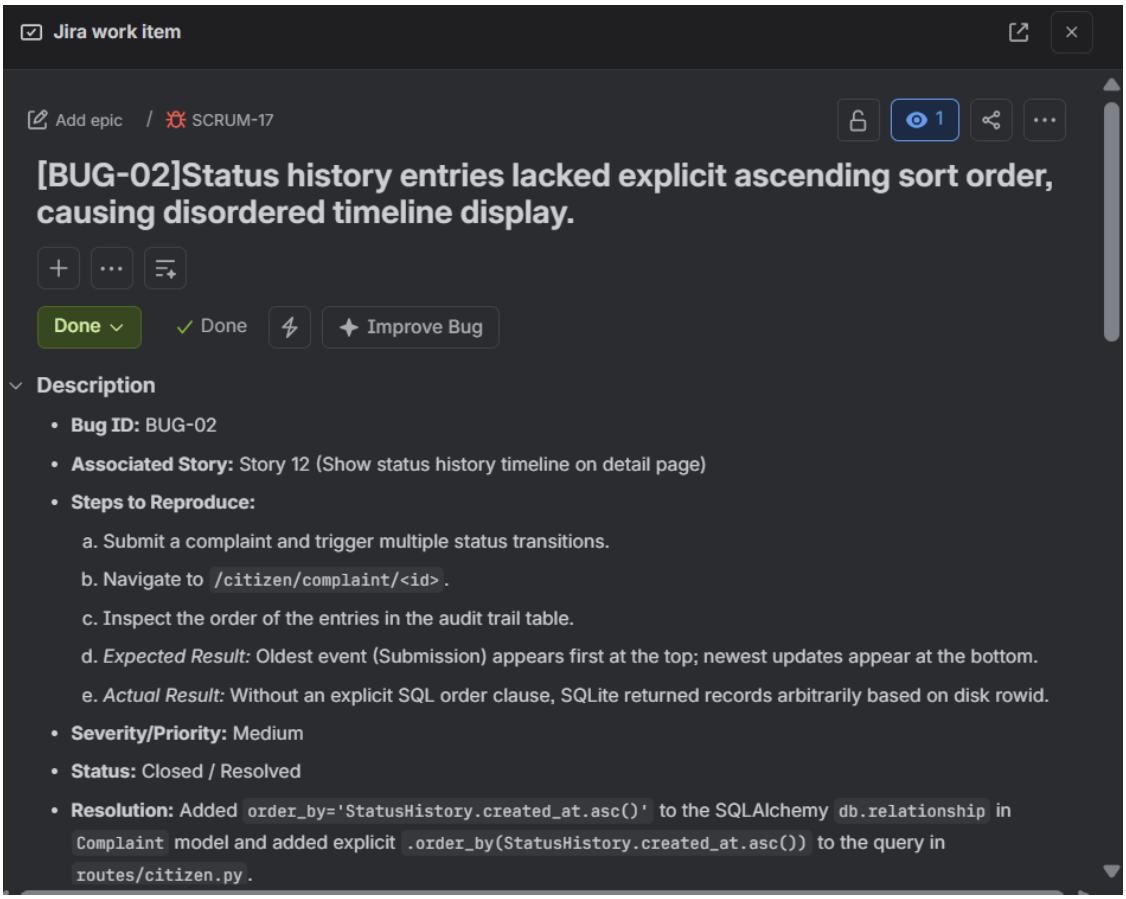


Figure 30: Bug 2

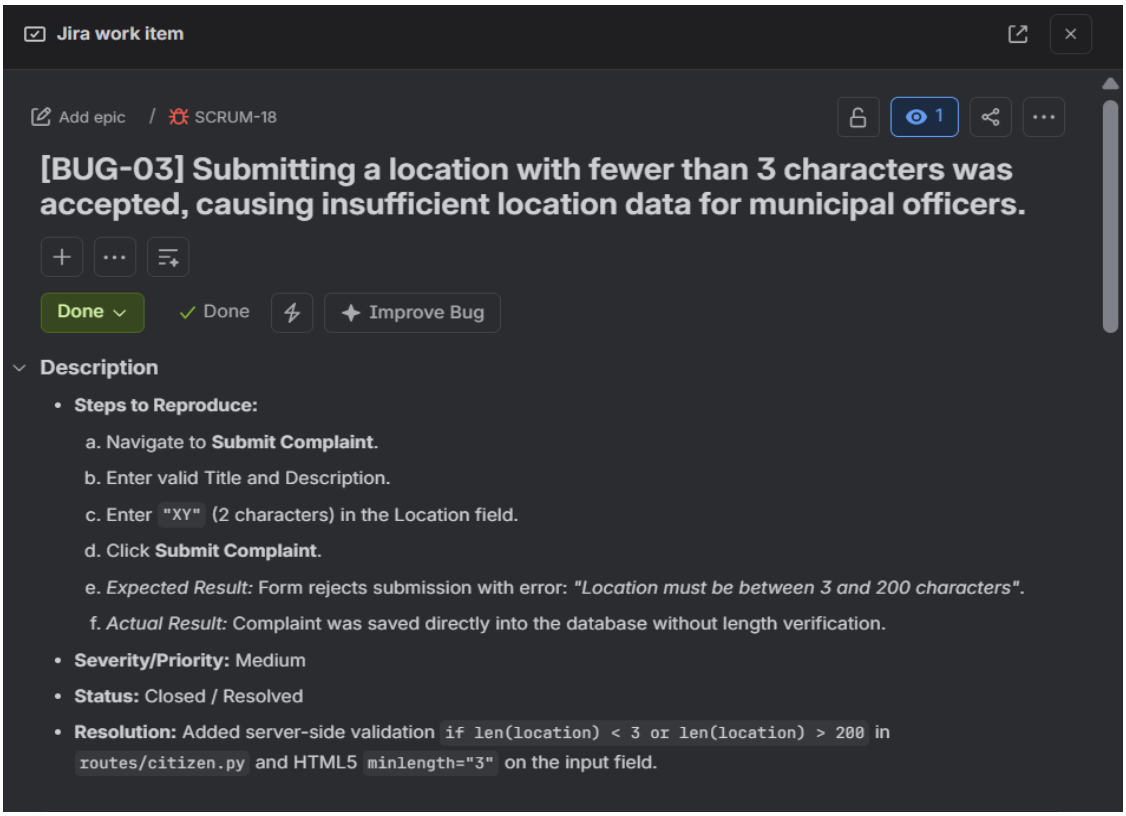


Figure 31: Bug 3

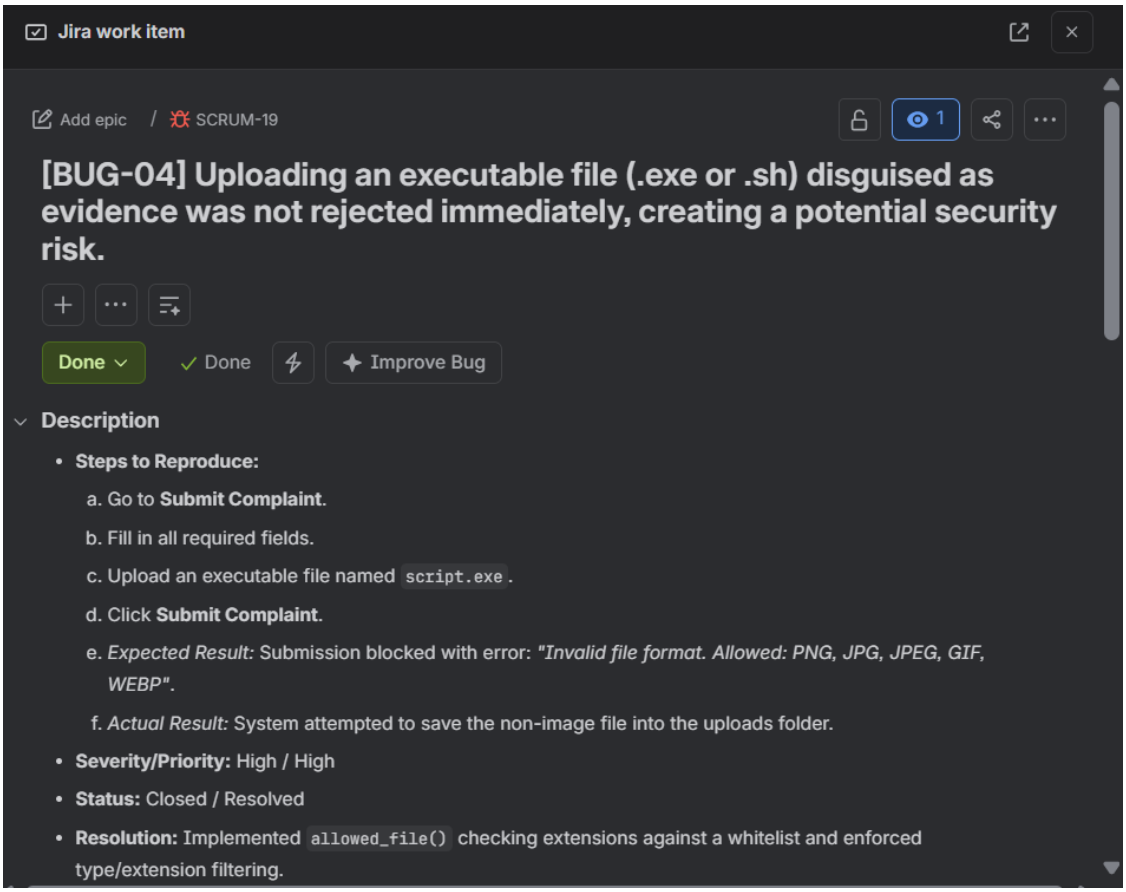


Figure 32: Bug 4



Figure 33: Bug 5

Retest: The test case was retested after fix: Complaint #1 retrieved when searching with whitespace. Pass.

Four additional defects were identified and resolved during development: [BUG-02] Missing chronological sort in status history (resolved by adding explicit order_by(created_at.asc())); [BUG-03] Location under 3 characters accepted (resolved by adding len(location) < 3 validation); [BUG-04] Executable file upload not filtered (resolved by checking allowed_file() extension whitelist); and [BUG-05] Cross-citizen complaint viewing via URL tampering (resolved by enforcing ownership check returning HTTP 403 Forbidden). All defects were verified through automated retests with zero regressions.

12. CONCLUSION

Two modules of the AI-Powered Civic Complaint Management System were implemented: Complaint Submission and Complaint Tracking. The project was planned and tracked in JIRA using epics, user stories, sub-tasks and two sprints. Eighteen test cases were designed and run using functional testing, validation testing, boundary value analysis, equivalence partitioning, unit testing and integration testing, together with 18 automated tests. Defects found during testing were logged in JIRA, fixed and retested.
Machine learning image classification, geospatial mapping, and citizen feedback ratings are left for future work. The system currently auto-assigns complaints to the designated municipal officer and maintains full lifecycle transparency.